

Design and Analysis of Algorithms - II

Answer 1(b)**(i) Quadratic probing**

$$h(k, i) = (k \bmod 11 + i + 3i^2) \bmod 11, \quad i = 0, 1, 2, \dots$$

Key	Probe sequence until insertion	Final slot
10	10	10
22	0	0
31	9	9
4	4	4
15	4, 8	8
28	6	6
17	6, 10, 9, 3	3
88	0, 4, 3, 8, 8, 3, 4, 0, 2	2
59	4, 8, 7	7

The repeated locations in the probe sequence for 88 occur because the given quadratic constants do not generate a new slot at every value of i .

Index	0	1	2	3	4	5	6	7	8	9	10
Value	22	–	88	17	4	–	28	59	15	31	10

(ii) Double hashing

$$h(k, i) = (h_1(k) + ih_2(k)) \bmod 11,$$

where

$$h_1(k) = k \bmod 11, \quad h_2(k) = 1 + (k \bmod 10).$$

Key	Probe sequence until insertion	Final slot
10	10	10
22	0	0
31	9	9
4	4	4
15	4, 10, 5	5
28	6	6
17	6, 3	3
88	0, 9, 7	7
59	4, 3, 2	2

Index	0	1	2	3	4	5	6	7	8	9	10
Value	22	–	59	17	4	15	28	88	–	31	10

Double hashing produces a much more varied probe sequence and avoids the long repeated pattern seen in the quadratic case.

Question 1(c) - 4 marks

Prove that the expected number of probes in an unsuccessful search is at most

$$\frac{1}{1 - \alpha}$$

for an open-address hash table with load factor $\alpha = n/m < 1$, assuming uniform hashing.

Answer 1(c)

Let X be the number of probes performed in an unsuccessful search. Under uniform hashing, the probability that the first probed location is occupied is $n/m = \alpha$. The probability that the first i probed positions are all occupied is

$$\frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-i+1}{m-i+1} \leq \left(\frac{n}{m}\right)^i = \alpha^i.$$

Hence

$$\Pr(X > i) \leq \alpha^i.$$

Using the tail-sum formula for a positive integer-valued random variable,

$$\mathbb{E}[X] = \sum_{i \geq 0} \Pr(X > i) \leq \sum_{i \geq 0} \alpha^i.$$

Since $0 \leq \alpha < 1$, the geometric series gives

$$\boxed{\mathbb{E}[X] \leq \frac{1}{1-\alpha}}.$$

Thus, for example, at $\alpha = 0.5$ the expected unsuccessful-search cost is at most 2 probes.

Question 2(a) - 3 marks

Define the state-space tree. Describe the features of a node of a state-space tree in the context of backtracking.

Answer 2(a)

A **state-space tree** is a rooted tree that represents all candidate states/partial solutions that can be generated while solving a combinatorial problem.

Node features in backtracking:

- **Root:** empty or initial solution.
- **Level:** the number of decisions already made.
- **State/partial solution:** choices made from the root to that node.
- **Children:** possible next choices.
- **Promising node:** may still lead to a feasible/optimal solution, so it is expanded.
- **Non-promising node:** violates a constraint or cannot lead to a solution, so it is pruned.
- **Solution leaf:** a complete feasible solution.

Backtracking performs a depth-first exploration and returns to an earlier node whenever the current node becomes non-promising.

Question 2(b) - 5 marks

Design a linear-time algorithm that finds a solution to the n -queens problem for any $n \geq 4$.

Answer 2(b)

A constructive $\mathcal{O}(n)$ method outputs the column positions of the queens directly. Let

$$E = (2, 4, 6, \dots), \quad O = (1, 3, 5, \dots)$$

be the even and odd column lists up to n . The i -th output number is the column of the queen in row i .

1. If $n \bmod 6 \notin \{2, 3\}$, output E followed by O .
2. If $n \bmod 6 = 2$, output E , then in O swap 1 and 3 and move 5 to the end.
3. If $n \bmod 6 = 3$, move 2 to the end of E ; move 1 and 3 to the end of O ; then output E followed by O .

Examples

$$n = 8 : \quad [2, 4, 6, 8, 3, 1, 7, 5],$$

$$n = 9 : \quad [4, 6, 8, 2, 5, 7, 9, 1, 3].$$

Each list contains every column exactly once, and the construction avoids equal diagonal differences. Creating and printing the two lists takes $\boxed{\mathcal{O}(n)}$ time.

Question 2(c) - 6 marks

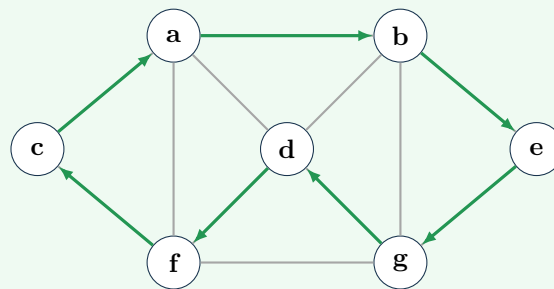
Apply backtracking to find a Hamiltonian circuit in the given graph.

Answer 2(c)

One successful backtracking path is

$$\boxed{c \rightarrow a \rightarrow b \rightarrow e \rightarrow g \rightarrow f \rightarrow c}.$$

It visits every vertex $\{a, b, c, d, e, f, g\}$ exactly once and returns to the start.



Backtracking idea. Start at c . Add an adjacent unvisited vertex. If a partial path has no unused adjacent vertex that can eventually return to c , undo the last choice and try another neighbor. The path above completes successfully, so search can stop if only one Hamiltonian circuit is required.

Question 3(a) - 4 marks

Describe the best-first branch-and-bound approach with a suitable example.

Answer 3(a)

In **best-first branch and bound**, all live nodes are kept in a priority queue. The algorithm always expands the live node with the most promising bound.

For maximization:

choose largest upper bound (UB), $UB \leq \text{bestValue} \Rightarrow \text{prune}$.

For minimization:

choose smallest lower bound (LB), $LB \geq \text{bestValue} \Rightarrow \text{prune}$.

Small maximization example. Suppose live nodes have upper bounds 92, 75 and 88 while the best feasible solution found so far has value 80. Best-first expands the node with UB 92 first; the node with UB 75 is immediately pruned because it cannot beat 80. A priority queue makes the next-node selection efficient.

General steps:

1. Create the root and compute its bound.
2. Remove the best-bound live node from the priority queue.
3. Branch into children.
4. Reject infeasible children.
5. Update the incumbent (best complete feasible solution).
6. Insert only children whose bound can still improve the incumbent.
7. Repeat until no live node remains.

Question 3(b) - 2 marks

Give an example of a best-case input for an assignment problem in the context of branch and bound.

Answer 3(b)

For a minimization assignment problem, a best-case input is one where the optimal assignment is immediately obvious and every alternative branch gets a much larger lower bound. For example,

$$C = \begin{bmatrix} 0 & 100 & 100 \\ 100 & 0 & 100 \\ 100 & 100 & 0 \end{bmatrix}.$$

The diagonal assignment ($1 \rightarrow 1, 2 \rightarrow 2, 3 \rightarrow 3$) has cost 0. Once this complete solution is found, every branch containing any off-diagonal assignment has lower bound at least 100, so all such branches are pruned immediately. Hence very few nodes are expanded.

Question 3(c) - 8 marks

Solve the following 0/1 knapsack problem using branch and bound.

Item	Weight	Value (BDT)	p_i/w_i
1	10	100	10
2	7	63	9
3	8	56	7
4	4	12	3

Capacity $W = 16$.

Answer 3(c)

The items are already in decreasing profit/weight-ratio order:

$$1 (10) \rightarrow 2 (9) \rightarrow 3 (7) \rightarrow 4 (3).$$

Bound used in this tree. If a node has current weight w , current profit p , and the best ratio among the *still undecided* items is r , then

$$UB = p + (W - w)r.$$

At a leaf, there is no undecided item, so $UB = p$. If $w > W$, the node is infeasible.

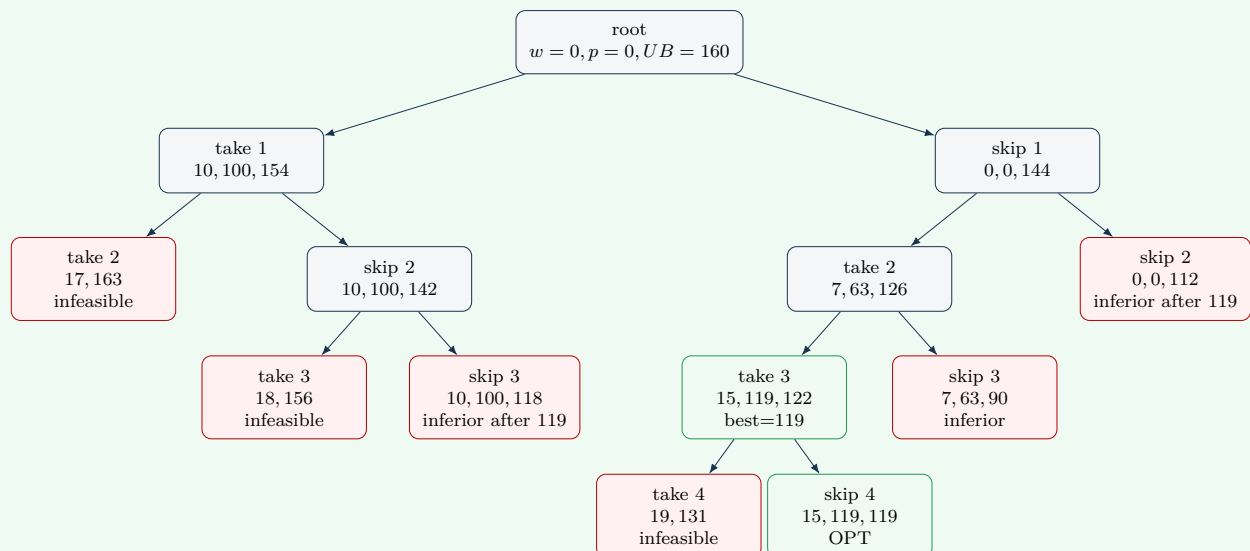
Important UB calculations:

$$\begin{aligned} UB(\emptyset) &= 0 + 16(10) = 160, \\ UB(1) &= 100 + (16 - 10)(9) = 154, \\ UB(0) &= 0 + 16(9) = 144, \\ UB(10) &= 100 + (16 - 10)(7) = 142, \\ UB(01) &= 63 + (16 - 7)(7) = 126, \\ UB(00) &= 0 + 16(7) = 112, \\ UB(100) &= 100 + (16 - 10)(3) = 118, \\ UB(011) &= 119 + (16 - 15)(3) = 122, \\ UB(010) &= 63 + (16 - 7)(3) = 90. \end{aligned}$$

Best-first search:

Path	Weight	Profit	UB	Action
root	0	0	160	expand
1	10	100	154	best becomes 100
0	0	0	144	live
11	17	163	–	infeasible
10	10	100	142	live
01	7	63	126	live
00	0	0	112	live
101	18	156	–	infeasible
100	10	100	118	live, later inferior
011	15	119	122	best becomes 119
010	7	63	90	inferior
0111	19	131	–	infeasible
0110	15	119	119	feasible leaf / optimum

Here 1 means “take the current item” and 0 means “skip it.”



Why pruning is valid. Once node 011 is reached,

$$w = 7 + 8 = 15, \quad p = 63 + 56 = 119,$$

so $\text{bestValue} = 119$. Then any live node with $UB \leq 119$ is inferior. Thus nodes 100 (UB 118), 00 (UB 112), and 010 (UB 90) cannot improve the answer.

Therefore,

take items 2 and 3,

weight = 15,

maximum value = BDT 119.

Question 4(a) - 4 marks

Write any string-matching algorithm that, given a pattern P , finds its occurrences in a text T . Analyze the complexity of the written algorithm in detail.

Answer 4(a)

A suitable answer is the **Knuth-Morris-Pratt (KMP)** algorithm.

Let $m = |P|$, $n = |T|$, and let $\pi[q]$ be the length of the longest proper prefix of $P[0..q]$ that is also a suffix.

```
KMP-MATCH(T, P)
  pi = PREFIX-FUNCTION(P)
  q = 0
  for i = 0 to n-1
    while q > 0 and P[q] != T[i]
      q = pi[q-1]
    if P[q] == T[i]
      q = q + 1
    if q == m
      report occurrence at i-m+1
      q = pi[m-1]
```

Complexity.

- Prefix-function computation: $\mathcal{O}(m)$.
- Text scan: $\mathcal{O}(n)$. Although the while-loop can execute several times at one position, q can increase only n times overall and each fallback decreases it, giving linear amortized work.
- Total time: $\mathcal{O}(n + m)$.
- Extra space: $\mathcal{O}(m)$ for the prefix array.

KMP never moves the text pointer backward.

Question 4(b) - 6 marks

You are given a pattern $P = \text{abbaab}$ and a text $T = \text{abaabbaabbbababa}$. Apply the KMP string-matching algorithm to find the occurrences of P in T . Write the prefix function/array and simulate the solution.

Answer 4(b)

Prefix function for $P = \text{abbaab}$.

Index i	0	1	2	3	4	5
$P[i]$	a	b	b	a	a	b
$\pi[i]$	0	0	0	1	1	2

Thus

$$\pi = [0, 0, 0, 1, 1, 2].$$

A compact KMP trace is shown below. q is the number of currently matched pattern characters after processing $T[i]$; when a full match is found, KMP reports it and resets q using π .

i	$T[i]$	Resulting q	Note
0	a	1	match a
1	b	2	match ab
2	a	1	fallback $2 \rightarrow 0$, then match a
3	a	1	fallback $1 \rightarrow 0$, then match a
4	b	2	match
5	b	3	match
6	a	4	match
7	a	5	match
8	b	2	full match P at start 3; reset $q = \pi[5] = 2$
9	b	3	continue
10	b	0	fallback $3 \rightarrow 0$, mismatch
11	a	1	match
12	b	2	match
13	a	1	fallback $2 \rightarrow 0$, then match a
14	b	2	match
15	a	1	fallback $2 \rightarrow 0$, then match a

The occurrence is

$$T[3..8] = \text{abbaab}.$$

Therefore

$$\text{one occurrence at 0-based shift 3}$$

or equivalently at the $\boxed{\text{4th character}}$ in 1-based indexing.

Question 4(c) - 4 marks

Given $S = \text{caabcba}$, write an algorithm to remove the minimum number of characters so that S becomes a palindrome. Show the resultant string.

Answer 4(c)

The minimum-deletion problem is equivalent to finding a **longest palindromic subsequence (LPS)**.

$$\text{minimum deletions} = n - \text{LPS}(S).$$

A standard dynamic-programming recurrence is

$$L[i][j] = \begin{cases} 1, & i = j, \\ 2 + L[i+1][j-1], & S[i] = S[j], \\ \max(L[i+1][j], L[i][j-1]), & S[i] \neq S[j]. \end{cases}$$

Fill the table by increasing substring length and reconstruct one LPS by walking backward through the table.

For

$$S = \text{c a a b c b a}, \quad n = 7,$$

one LPS is

`abcba`

of length 5. Thus

$$7 - 5 = 2 \text{ deletions}.$$

For example, delete the initial `c` and the second extra `a`; the resultant string is

`abcba`,

which is a palindrome.

The DP runs in $\mathcal{O}(n^2)$ time and $\mathcal{O}(n^2)$ space (or $\mathcal{O}(n)$ space if only the count is needed).

Question 5(a) - 7 marks

Explain the Marker's Algorithm. Apply the algorithm to the following pseudocode to extract the results.

```
1. def func(a, b):
2.     c = a + b
3.     d = 2 * c - a
4.     e = 10
5.     f = 2 * d
6.     print(f)
```

Answer 5(a)

In the context of this pseudocode, **Marker's Algorithm** is a backward dependence-marking method used to identify statements that contribute to the required output. (This is different from the randomized paging "marking algorithm.")

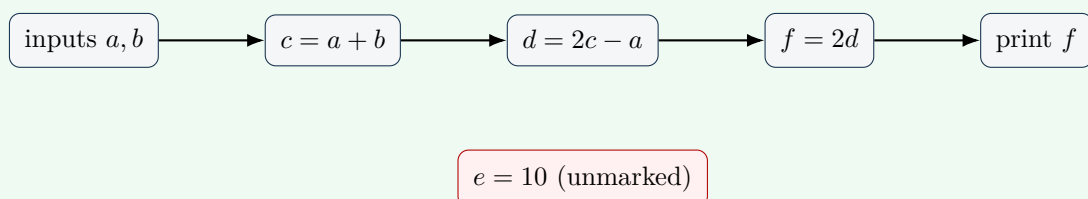
Procedure.

1. Mark the output/use statement(s).
2. If a marked statement uses a variable, mark the statement that defines that variable.
3. Recursively mark all variables and definitions needed by those marked statements.
4. Any unmarked assignment is irrelevant/dead for the requested output and can be removed.

Apply it backward from `print(f)`:

$$\text{print}(f) \Rightarrow f \Rightarrow d \Rightarrow (c, a) \Rightarrow (a, b).$$

So the marked lines are 2, 3, 5 and 6 (plus the function header). Line 4, `e = 10`, is never used and remains unmarked.



The extracted/optimized program is

```
def func(a, b):
    c = a + b
    d = 2 * c - a
    f = 2 * d
    print(f)
```

Symbolically,

$$d = 2(a + b) - a = a + 2b, \quad f = 2d = \boxed{2a + 4b}.$$

Question 5(b) - 7 marks

Solve the system of congruences using the Chinese Remainder Theorem (CRT):

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}, \quad x \equiv 2 \pmod{7}.$$

Find the smallest positive integer satisfying all three.

Answer 5(b)

The moduli 3, 5 and 7 are pairwise coprime, so CRT applies.

$$N = 3 \cdot 5 \cdot 7 = 105.$$

Define

$$N_1 = 105/3 = 35, \quad N_2 = 105/5 = 21, \quad N_3 = 105/7 = 15.$$

Find inverses:

$$\begin{aligned} 35 &\equiv 2 \pmod{3}, & 2^{-1} &\equiv 2 \pmod{3}, \\ 21 &\equiv 1 \pmod{5}, & 1^{-1} &\equiv 1 \pmod{5}, \\ 15 &\equiv 1 \pmod{7}, & 1^{-1} &\equiv 1 \pmod{7}. \end{aligned}$$

Hence

$$\begin{aligned} x &\equiv 2(35)(2) + 3(21)(1) + 2(15)(1) \pmod{105} \\ &= 140 + 63 + 30 \\ &= 233 \\ &\equiv 23 \pmod{105}. \end{aligned}$$

Therefore the smallest positive solution is

$$\boxed{x = 23}.$$

Check: $23 \bmod 3 = 2$, $23 \bmod 5 = 3$, $23 \bmod 7 = 2$.

Question 6(a) - 2 marks

State the decision version of the Independent Set and Vertex Cover problems.

Answer 6(a)

Independent Set (decision). Given a graph $G = (V, E)$ and integer k , does there exist a set $S \subseteq V$ with $|S| \geq k$ such that no two vertices in S are adjacent?

Vertex Cover (decision). Given a graph $G = (V, E)$ and integer k , does there exist a set $C \subseteq V$ with $|C| \leq k$ such that every edge in E has at least one endpoint in C ?

Question 6(b) - 6 marks

Establish a relationship between Independent Set and Vertex Cover that transforms an input instance of one problem into an input instance of the other. State the relationship as a lemma and prove it.

Answer 6(b)

Lemma. For any graph $G = (V, E)$, a set $S \subseteq V$ is an independent set if and only if $V - S$ is a vertex cover.

Proof ($\Rightarrow$). Suppose S is independent. No edge can have both endpoints in S . Therefore every edge has at least one endpoint in $V - S$. Hence $V - S$ is a vertex cover.

Proof ($\Leftarrow$). Suppose $V - S$ is a vertex cover. If two vertices of S were adjacent, the edge between them would have neither endpoint in $V - S$, contradicting that $V - S$ covers every edge. Hence S is independent.

Therefore,

$$G \text{ has an independent set of size } k \iff G \text{ has a vertex cover of size } |V| - k.$$

The reduction simply maps

$$(G, k)_{IS} \mapsto (G, |V| - k)_{VC},$$

which is computable in polynomial (indeed linear) time.

Question 6(c) - 6 marks

Give a 2-approximation algorithm for Vertex Cover. Prove that the approximation ratio is 2.

Answer 6(c)

Use a **maximal matching**.

```

APPROX-VERTEX-COVER( $G=(V,E)$ )
  C = empty set
  E' = E
  while E' is not empty
    choose any edge (u,v) in E'
    add u and v to C
    delete from E' every edge incident to u or v
  return C

```

The chosen edges form a matching M , because after selecting (u, v) all edges incident to u or v are deleted. The returned set contains both endpoints of every selected matching edge, so

$$|C| = 2|M|.$$

Any vertex cover C^* must contain at least one endpoint of each edge in the matching. Since matching edges have no shared endpoints,

$$|C^*| \geq |M|.$$

Therefore

$$|C| = 2|M| \leq 2|C^*|.$$

Thus

$$\frac{|C|}{|C^*|} \leq 2,$$

so the algorithm is a 2-approximation.

Question 7(a) - 4 marks

Ski rental problem: renting costs BDT 500 per day and buying costs BDT 1500. The number of skiing days is unknown. Mention the features of an online algorithm and define/analyze the competitive ratio using three strategies for this ski-rental problem.

Answer 7(a)**Features of an online algorithm.**

- Input arrives over time rather than being known completely in advance.
- A decision must be made using only the past and present information.
- Future requests/events are unknown.
- Performance is compared with an optimal offline algorithm OPT, which knows the entire future.
- A deterministic online algorithm is c -competitive if

$$A(\sigma) \leq c \text{OPT}(\sigma) + b$$

for all input sequences σ , where b is a constant independent of sequence length.

Let the actual trip length be N days. Offline optimum is

$$\text{OPT}(N) = \min(500N, 1500).$$

Strategy 1: Always rent. Cost = $500N$. For large N ,

$$\frac{500N}{1500} = \frac{N}{3} \rightarrow \infty.$$

So this strategy has no constant competitive ratio.

Strategy 2: Buy immediately. Cost = 1500. For $N = 1$, $\text{OPT} = 500$, so the worst ratio is

$$\boxed{1500/500 = 3}.$$

Strategy 3: Rent for two days, then buy at the start of day 3 if skiing continues.

- If $N \leq 2$, online cost = $500N = \text{OPT}$.
- If $N \geq 3$, online cost = $2(500) + 1500 = 2500$, while $\text{OPT} = 1500$.

Thus

$$\boxed{c = \frac{2500}{1500} = \frac{5}{3}}.$$

For these discrete costs, the threshold strategy is much better than always rent or always buy.

Question 7(b)(i) - 4 marks

Consider the following linear list-search problem. Consider a case in which the accessed item is moved two positions forward and the request sequence is only for the last item of the list. Calculate the competitive ratio.

Answer 7(b)(i)

Let the list length be $n = 2m$ for simplicity and repeatedly request the same item that initially occupies position n . Under the “move two positions forward” rule, its successive access costs are

$$2m, 2m - 2, 2m - 4, \dots, 2.$$

After m requests, the online cost is

$$A = m(m + 1).$$

An offline optimum, knowing that the same item will be requested repeatedly, can move it to the

front after the first access under the standard list-update model. Its cost is at most

$$\text{OPT} \leq 2m + (m - 1) = 3m - 1.$$

Therefore

$$\frac{A}{\text{OPT}} \geq \frac{m(m+1)}{3m-1} = \Theta(m) = \Theta(n).$$

So this “move only two positions” rule is

not constant-competitive; its competitive ratio is unbounded as $n \rightarrow \infty$.

This illustrates why moving an accessed item only a fixed small distance can be much worse than Move-To-Front.

Question 7(b)(ii) - 6 marks

Prove that no deterministic online algorithm has competitive ratio better than 2 for the linear list-search problem.

Answer 7(b)(ii)

Use an adversary. Before every request, the adversary asks for the item currently in the **last position** of the online algorithm's list. Thus an online algorithm A pays cost n on every request.

For the same request sequence, consider all $n!$ fixed list permutations. For any requested item, its average position over all permutations is

$$\frac{1 + 2 + \cdots + n}{n} = \frac{n+1}{2}.$$

Therefore there exists at least one fixed ordering whose total cost on the whole request sequence is at most $(n+1)/2$ per request on average. Since the optimal offline algorithm can do at least as well as this fixed ordering,

$$\text{OPT} \leq \frac{n+1}{2} \times (\text{number of requests}).$$

But A pays

$$n \times (\text{number of requests}).$$

Hence the ratio can be forced to at least

$$\frac{n}{(n+1)/2} = \frac{2n}{n+1}.$$

As $n \rightarrow \infty$,

$$\frac{2n}{n+1} \rightarrow 2.$$

Therefore no deterministic online list-update algorithm can have competitive ratio strictly smaller than 2 (up to the usual additive constant):

$$c \geq 2.$$

Last-Minute Revision Sheet

Hashing

Primary clustering is the growth of contiguous runs under linear probing. Quadratic probing reduces primary clustering; double hashing gives different step sizes. Under uniform hashing, unsuccessful-search probes are at most $1/(1 - \alpha)$ in expectation.

Backtracking and Branch-and-Bound

Backtracking prunes non-promising partial solutions. Best-first B&B expands the live node with best bound. Maximization: prune if $UB \leq$ incumbent. Minimization: prune if $LB \geq$ incumbent.

KMP

Prefix function stores longest proper prefix that is also suffix. On mismatch, repeatedly set $q = \pi[q - 1]$; never move the text pointer backward. Time $\mathcal{O}(n + m)$.

CRT

For pairwise-coprime moduli m_i : $N = \prod m_i$, $N_i = N/m_i$, choose $y_i = N_i^{-1} \pmod{m_i}$, then $x \equiv \sum a_i N_i y_i \pmod{N}$.

Independent Set / Vertex Cover

$$S \text{ independent} \iff V - S \text{ vertex cover.}$$

A maximal matching gives a 2-approximate vertex cover by taking both endpoints of every matching edge.

Online Algorithms

Competitive analysis compares online cost to offline OPT. Ski rental here: buy immediately ratio 3; rent two days then buy ratio $5/3$. For deterministic list update, no competitive ratio below 2 is possible.